SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME:Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5
4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and	5

	reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5
9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by	5

	analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	

Question 1: ATM Withdrawal Module (Black-Box Reverse Engineering)

1. Understanding of the System

Only input-output behaviour of the ATM withdrawal module is available. Because the raw observations are not attached to the task, the following behaviour is **assumed** as typical observed behaviour and is used as the basis of the reverse engineering.

Observed input	Observed output
Amount = 500, balance 5,000	Cash dispensed, balance becomes 4,500
Amount = 50 or 99	"Invalid amount. Minimum withdrawal is Rs.100"
Amount = 550	"Enter amount in multiples of 100"
Amount = 10,000 accepted; 10,001 rejected	"Per-transaction limit exceeded"
Amount > balance	"Insufficient funds"
Total of the day reaches 25,000	"Daily limit reached"
3 wrong PIN entries	"Card blocked"

2. Decomposition into Functional Components

Component	Function	Inputs	Possible outputs
C1 Authentication	Validate card status and PIN, count attempts	Card, PIN	Access / retry / card blocked
C2 Input validation	Numeric, positive, multiple of 100	Amount	"Invalid amount"
C3 Limit checker	Per-transaction (min 100, max 10,000) and daily limit (25,000)	Amount, daily total	"Limit exceeded" / "Daily limit reached"
C4 Balance validator	Amount <= available balance	Amount, balance	"Insufficient funds"
C5 Cash dispenser	Check ATM cash and denominations, dispense	Amount, cash cassette	Dispense / "Cash unavailable"
C6 Post-processing	Update balance, print receipt, write log	Transaction result	New balance, receipt

3. Equivalence Classes and Boundary Conditions

Input	Valid class	Invalid classes
Amount	100 <= amt <= 10,000 and amt % 100 = 0	amt < 100 (0, negative); amt > 10,000; not multiple of 100; non-numeric / empty
Balance vs amount	balance >= amt	balance < amt
Daily total	today + amt <= 25,000	today + amt > 25,000
Card / PIN	Active card, correct PIN	Blocked / expired card; wrong PIN (1st, 2nd, 3rd)
ATM cash	cash >= amt	cash < amt

Boundary values: amount 0, 99, 100, 101, 9,900, 10,000, 10,001; balance = amt-1, amt, amt+1; daily total 24,900 / 25,000 / 25,100; PIN attempts 2, 3 and 4.

4. Inferred Black-Box Techniques and Adequacy

Technique	Evidence in observed behaviour	Adequacy
Equivalence Partitioning	One typical valid (500) and invalid (50, 550) value per rule	Adequate for single fields
Boundary Value Analysis	Limits 100 and 10,000 tested at the edges	Partial: daily limit and balance edges missing
Error guessing	Negative / alphabetic input rejected	Informal and incomplete
Decision table	Not evident: combined conditions untested	Gap
State transition	Not evident: PIN-attempt sequence untested	Gap

Gaps: no combination of conditions (e.g. amount valid but balance low and daily limit exceeded), no error-priority check, no card-block state testing, no cash-unavailable or hardware-failure test, no concurrency or timeout tests.

5. Reconstructed Test Cases

TC	Input / condition	Technique	Expected result
1	500, balance 5,000	EP valid	Dispense, balance 4,500
2	99	BVA	"Invalid amount"
3	100	BVA	Success
4	10,000 (balance sufficient)	BVA	Success
5	10,001	BVA	"Limit exceeded"
6	550	EP	"Multiples of 100 only"
7	0 and -500	EP/BVA	"Invalid amount"
8	"abc" / empty	Error guessing	Input rejected
9	Balance 1,000, withdraw 1,000	BVA	Success, balance 0
10	Balance 1,000, withdraw 1,100	BVA	"Insufficient funds"
11	Daily used 24,900, withdraw 100	BVA	Success
12	Daily used 25,000, withdraw 100	BVA	"Daily limit reached"
13	Amount 10,000, balance 5,000	Decision table	Defined error priority applied
14	Wrong PIN x3	State transition	Card blocked
15	Wrong PIN x2 then correct	State transition	Access, counter reset
16	Blocked / expired card	EP	Rejected
17	ATM cash 400, request 500	EP	"Cash unavailable"
18	Network failure mid-transaction	Error guessing	Rollback, no debit
19	Two simultaneous withdrawals	Concurrency	Balance consistent

Decision table for combined conditions

Rule	Amount valid	Balance OK	Daily OK	Cash OK	Action
R1	Y	Y	Y	Y	Dispense
R2	N	-	-	-	Invalid amount
R3	Y	N	-	-	Insufficient funds

Rule	Amount valid	Balance OK	Daily OK	Cash OK	Action
R4	Y	Y	N	-	Daily limit reached
R5	Y	Y	Y	N	Cash unavailable

6. Conclusion

The original testing was mostly EP and BVA on the amount field. The improved suite adds decision-table and state-based tests, closing gaps in rule interaction, security (PIN lock) and failure handling.

Question 2: White-Box Reverse Engineering from Code and Traces

1. Understanding of the System

No test documentation exists, so the code logic is reconstructed from the given snippet and execution traces. The module computes an order total from item quantities and applies a discount. The snippet is assumed as follows (price = 100 per unit).

```
N1   total = 0; i = 0;
N2   while (i < n) {
N3       if (qty[i] <= 0) { i++; continue; }      // N4 skip invalid item
N5       total += qty[i] * price;
N6       i++;
     }
N7   if (total > 1000) {
N8       if (member) total *= 0.90;              // N9
         else        total *= 0.95;              // N10
     }
N11  return total;
```

Assumed execution traces available: T1 (n=0) gives path N1-N2-N7-N11; T2 (qty = 5, 3; non-member) gives total 800 and the loop runs twice; T3 (qty = 20; member) gives total 1,800 via the member discount.

2. Decomposition and Component Identification

Component	Nodes	Function
Initialisation	N1	Set accumulators
Loop controller	N2	Iterate through items
Item validator	N3, N4	Skip zero or negative quantity
Accumulator	N5, N6	Add line total, advance index
Discount engine	N7-N10	Tiered discount for totals over 1,000 (member or non-member)
Output	N11	Return result

3. Control Flow Graph

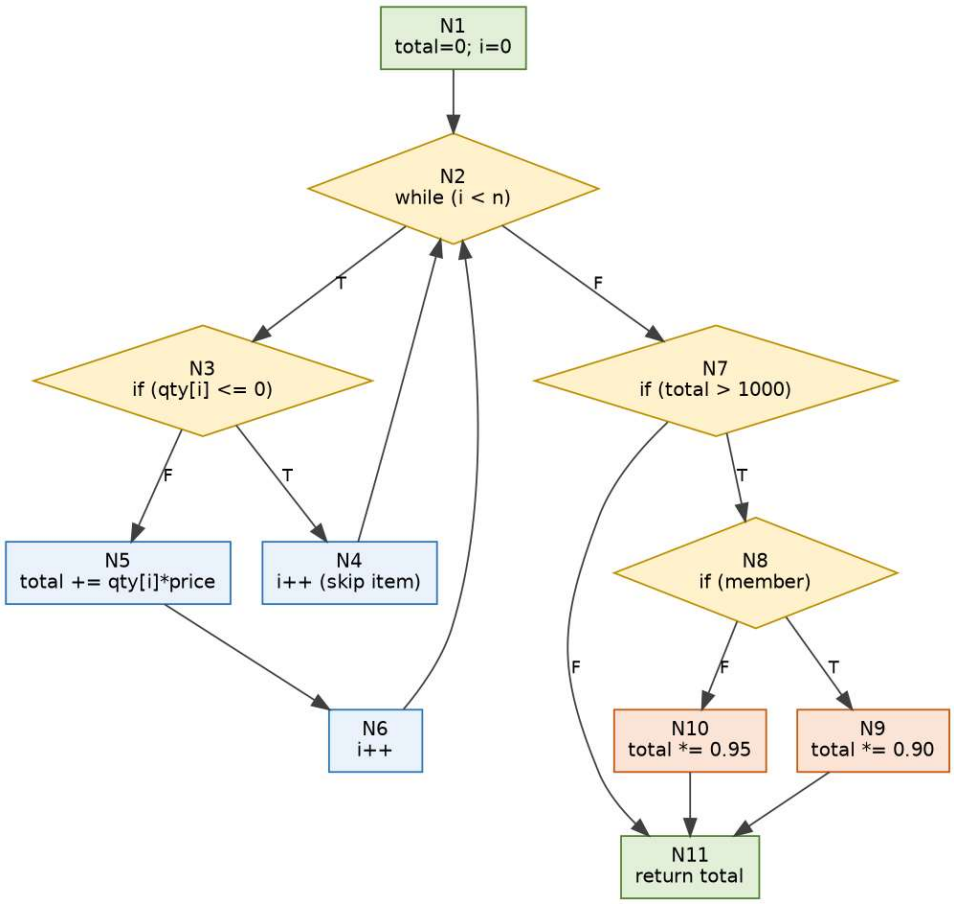


Figure 2.1 Control flow graph of the order module (diamonds are decision points)

4. Decision Points, Loops and Independent Paths

Decision points: N2 (loop condition), N3, N7, N8. **Loop:** N2 with back edges from N4 and N6.

Cyclomatic complexity: Edges E = 14, Nodes N = 11, so V(G) = E - N + 2 = **5** (equals number of predicate nodes + 1 = 4 + 1). Therefore 5 independent (basis) paths are required.

Path	Node sequence	Test data	Expected
P1	N1-N2-N7-N11	n=0	total 0
P2	N1-N2-N3-N5-N6-N2-N7-N11	qty=[5]	500
P3	N1-N2-N3-N4-N2-N7-N11	qty=[0]	0 (item skipped)
P4	N1-N2-N3-N5-N6-N2-N7-N8-N9-N11	qty=[20], member	1,800
P5	N1-N2-N3-N5-N6-N2-N7-N8-N10-N11	qty=[20], non-member	1,900

5. Evaluation of Existing Test Adequacy

Criterion	Covered by T1-T3	Result
Statement coverage	N1, N2, N3, N5, N6, N7, N8, N9, N11 (N4 and N10 missed)	9 / 11 = 82%
Branch (decision) coverage	While T/F, N3 false only, N7 T/F, N8 true only	6 / 8 = 75%
Basis path coverage	P1, P2-type, P4	3 / 5 = 60%
Loop coverage	0 and 2 iterations only; 1 iteration and n+1 not tested	Incomplete

6. Reconstructed Path Testing Strategy

- Execute all 5 basis paths (P1-P5) to reach 100% statement, branch and basis-path coverage.

- **Loop testing:** 0 iterations, 1 iteration, 2 iterations, typical n, n-1, n and n+1 (boundary of loop bound).
- **Mixed path:** qty=[0, 5, -1, 20] to exercise skip and accumulate alternately inside one run.
- **Boundary of the decision at N7:** total = 1000 (no discount) and total = 1001-1100 (discount applied).

TC	Input	Path covered	Expected
1	n=0	P1	0
2	[5]	P2	500
3	[0]	P3	0
4	[20], member	P4	1,800
5	[20], non-member	P5	1,900
6	[10] (total 1000)	N7 false boundary	1,000
7	[11], member (total 1100)	N7 true boundary	990
8	[0, 5, -1, 20], member	Mixed loop paths	2,250

Conclusion: The reconstructed suite gives 100% statement, branch and basis-path coverage and adds loop and boundary tests missing from the original traces.

Question 3: Web Login and Transaction System (Defect-Log Analysis)

1. Understanding of the System

A web application offers user login and fund-transaction features. Only defect logs and results exist, so the techniques used are inferred from defect patterns. The defect log below is **assumed** as representative.

ID	Defect observed	Pattern
D1	Password of exactly 8 characters rejected (minimum is 8)	Off-by-one at boundary
D2	Username with 51 characters accepted (maximum is 50)	Upper boundary
D3	Transfer of 0.00 accepted	Lower boundary of amount
D4	Account locked after 4th wrong attempt instead of 3rd	Counter logic / state
D5	Transfer succeeds when balance is low AND daily limit exceeded (wrong message shown)	Combined conditions
D6	Session still active after logout via Back button	Session state
D7	SQL special characters in username cause error page	Invalid class / injection
D8	Retry after timeout posts a duplicate debit	Unexecuted path (error branch)

2. Decomposition into Testable Components

Component	Function	Likely technique originally used
Login form	Validate username and password format	EP, BVA (D1, D2)
Authentication / lockout	Credential check, failed-attempt counter	State testing (partial) (D4)
Session manager	Create, expire, destroy session	Little testing (D6)
Transaction form	Amount, payee, validation	BVA partial (D3)
Transaction engine	Balance check, limits, debit/credit, rollback	Statement-level white box only (D5, D8)
Error handler	Messages, exceptions, retries	Error guessing (D7)

3. Inference of Techniques and Coverage Gaps

- **Black box used (likely):** equivalence partitioning with one value per class, and boundary value analysis on only some fields. Defects D1-D3 survived because boundaries were tested on one side only.
- **White box used (likely):** statement coverage only. Defects D5 and D8 sit in compound conditions and rarely executed error branches, which statement coverage does not force.
- **Not used:** decision tables (D5), state-transition testing (D4, D6), path testing of the retry loop (D8), security testing (D7).

4. Reconstructed Comprehensive Strategy

4.1 Equivalence partitioning and BVA

Field	Valid partition	Invalid partitions	Boundary values
Username	1-50 chars	empty; >50; illegal chars	0, 1, 2, 49, 50, 51
Password	8-20 chars with rules	<8; >20; no digit	7, 8, 9, 19, 20, 21
Amount	0.01 - balance and <= 50,000	<=0; >limit; >balance; non-numeric	0, 0.01, 0.02, 49,999.99, 50,000, 50,000.01

Field	Valid partition	Invalid partitions	Boundary values
Failed attempts	1-2	3 and above	2, 3, 4

4.2 Decision table for transaction approval

Rule	Logged in	Balance OK	Limit OK	Payee valid	Result
R1	Y	Y	Y	Y	Approve
R2	N	-	-	-	Redirect to login
R3	Y	N	-	-	Insufficient funds
R4	Y	Y	N	-	Limit exceeded
R5	Y	Y	Y	N	Invalid payee
R6	Y	N	N	-	Insufficient funds (priority rule)

4.3 Path testing (white box) of transaction engine

Decision nodes: session valid, amount valid, balance sufficient, limit OK, commit success, retry. Cyclomatic complexity V(G) = 7 (6 predicates + 1). Each of the 7 basis paths is tested, including the commit-failure then rollback path and the retry path (verifying no duplicate debit, which would have caught D8).

4.4 State-transition tests

States: Logged-out, Logged-in, Locked, Session-expired. Tests: 3 wrong passwords then lock; 2 wrong then correct; logout then Back button (D6); idle timeout then action; locked account with correct password.

5. Justification

Each defect class maps to a missing technique: boundary defects to two-sided BVA, combined-condition defects to decision tables, counter and session defects to state testing, retry and rollback defects to path testing, and injection defects to invalid-class security tests. Together they cover input, logic, state and error handling, which a single technique cannot.

Question 4: Legacy Inventory Management System (Log Analysis)

1. Understanding of the System

Only transaction logs exist for stock updates and low-stock alerts. Logs below are **assumed** representative samples.

Log	Item stock	Operation	Result	Alert
L1	100	Issue 20	SUCCESS, stock 80	None
L2	30	Issue 20	SUCCESS, stock 10	LOW STOCK (<= 10)
L3	30	Issue 19	SUCCESS, stock 11	None
L4	10	Issue 11	FAIL "Insufficient stock"	-
L5	10	Issue 10	SUCCESS, stock 0	OUT OF STOCK
L6	4,990	Receive 20	FAIL "Exceeds capacity 5,000"	-
L7	4,990	Receive 10	SUCCESS, stock 5,000	None
L8	50	Issue 0 / -5	FAIL "Invalid quantity"	-
L9	50	Issue 2.5	FAIL "Quantity must be whole number"	-

2. Inferred Business Rules and Hidden Boundaries

- Quantity must be a positive whole number (L8, L9).
- Issue allowed only if quantity <= current stock (L4 vs L5): boundary at stock = quantity.
- Low-stock alert triggers when resulting stock <= 10 (L2 vs L3): reorder level = 10.
- Out-of-stock alert when resulting stock = 0 (L5).
- Maximum capacity 5,000: receipt allowed only if stock + quantity <= 5,000 (L6, L7).

3. Equivalence Partitions

Variable	Valid partitions	Invalid partitions
Quantity	Positive integer	0, negative, decimal, non-numeric, blank
Issue vs stock	qty < stock (normal); qty = stock (empties)	qty > stock
Resulting stock (alert)	> 10 (no alert); 1-10 (low); 0 (out)	< 0 (rejected)
Receipt vs capacity	stock + qty <= 5,000	stock + qty > 5,000

4. Decomposition into Functional Units

Unit	Function
U1 Input validator	Type and range of quantity
U2 Stock checker	Compare quantity with stock / capacity
U3 Stock updater	Add or subtract, persist to database
U4 Alert engine	Evaluate resulting stock against reorder level and zero
U5 Logger	Write success or failure entry

5. Reconstructed Black-Box Test Design

TC	Stock	Operation	Technique	Expected
1	100	Issue 20	EP	Success, 80, no alert

TC	Stock	Operation	Technique	Expected
2	30	Issue 19	BVA (11)	Success, no alert
3	30	Issue 20	BVA (10)	Success, low-stock alert
4	30	Issue 21	BVA (9)	Success, low-stock alert
5	10	Issue 9	BVA (1 left)	Success, low-stock alert
6	10	Issue 10	BVA (0)	Success, out-of-stock alert
7	10	Issue 11	BVA (-1)	Fail, insufficient stock
8	50	Issue 0, -1, 2.5, "abc"	EP invalid	Fail, invalid quantity
9	4,990	Receive 10	BVA (5,000)	Success
10	4,990	Receive 11	BVA (5,001)	Fail, capacity
11	0	Issue 1	Edge	Fail, no stock
12	12	Two concurrent issues of 7	Concurrency	One succeeds, one fails

6. Justification

Test values sit exactly on each inferred threshold (10/11 for alert, 0/-1 for stock, 5,000/5,001 for capacity) because defects in legacy code concentrate at boundaries. EP covers each partition once, and the concurrency case covers the risk of lost updates typical in old systems. Inferred rules should be confirmed with the business owner.

Question 5: Library Management .exe with CPU/Memory Profile (Bloom Level 6)

1. Understanding of the System

Only a compiled executable and its stress-test profile are available, so the logic is inferred from resource behaviour. The profile below is **assumed**.

Time / action during stress test	CPU	Memory	Likely cause
Start-up	60% for 5 s	Rises to 150 MB	Loop loading N records
Search (repeated)	Spikes to 95% growing with catalogue size	Flat	Nested loop books x fields, O(n*m)
Checkout x 1000	Low (20%)	Steady increase, never released	Allocation per transaction, no free (leak)
Return with fines	Spikes to 80%	Flat	Loop per late day inside loop per loan
Report generation	100% for long period	Jumps by 400 MB	Loop over all transactions, appends to in-memory list

2. Mapping Execution Spikes to Logic

Spike	Inferred construct	White-box requirement
Search CPU	Nested loops + match decision	Loop testing: 0, 1, n, max iterations; both match outcomes
Fine calculation CPU	Loop in loop with overdue decision	Overdue true/false; 0, 1, many days; many loans
Checkout memory growth	Allocation path with missing release	Repeated execution of the issue path
Report CPU and memory	Single large loop with append	Max-size loop path; memory bound check

3. Conceptual Control Flow Graph

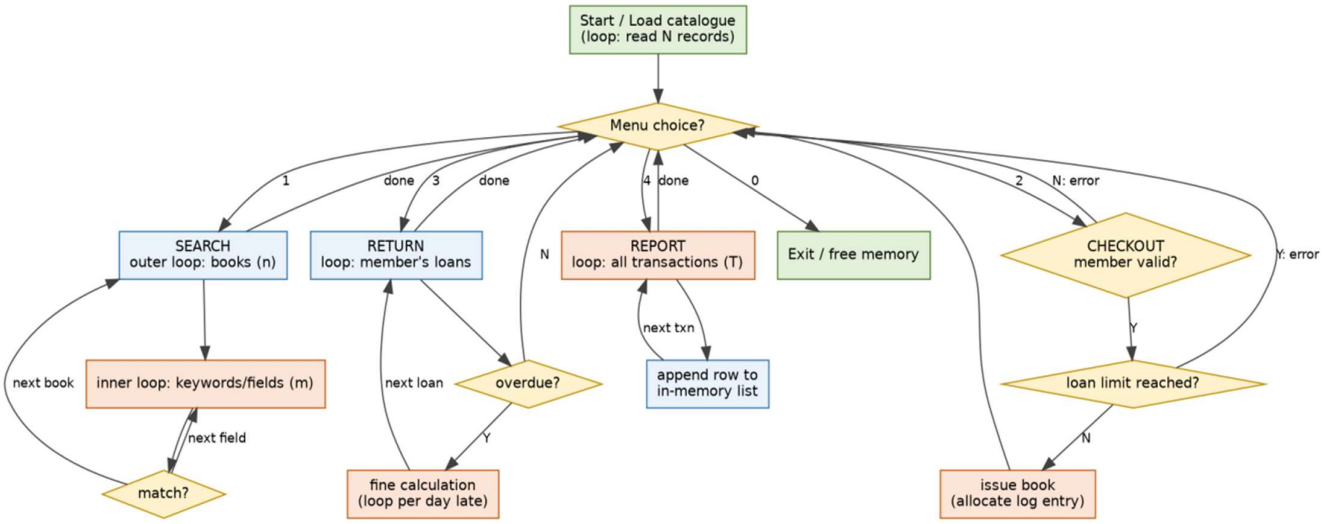


Figure 5.1 Conceptual CFG (orange nodes are the resource-intensive regions)

Decision clusters: menu (5-way), search match, member valid, loan limit, overdue. Loops: load loop, search outer and inner loops, return loop with fine loop, report loop.

4. Reconstructed Path Testing Strategy Targeting Heavy Paths

ID	Path targeted	Test	Pass criterion
PT1	Menu-Search-nested loop (no match)	10,000 books, 5 fields, worst case	Response time within limit, CPU returns to baseline
PT2	Search with match at first / last record	Best and worst case	Correct result both ends
PT3	Checkout valid path repeated	5,000 consecutive issues	Memory growth flat (no leak)
PT4	Checkout invalid member / limit reached	Error paths x 1000	No leak on error branches
PT5	Return, overdue, fine loop	Member with 50 loans, each 365 days late	No hang; correct fine
PT6	Report loop max size	1 million transactions	Memory under cap; graceful failure if exceeded
PT7	Mixed concurrent search + report	Parallel threads	No deadlock or crash

5. Impact on System Stability

- Detects memory leaks (PT3, PT4) before they cause out-of-memory crashes in long-running sessions.
- Exposes algorithmic inefficiency (PT1, PT5) so that indexing or early-exit logic can be introduced.
- Defines safe operating limits (PT6): maximum catalogue and transaction size.
- Combined with CPU/memory monitoring during path execution, the strategy shows which exact paths threaten stability, so fixes are targeted instead of general.

Question 6: Mobile Payment Gateway (State-Transition Testing)

1. Understanding of the System

JSON responses for Pending, Success, Reversal and Timeout states are analysed. Assumed sample responses:

```
{ "txnId":"T101", "status":"PENDING",  "amount":500, "retry":0 }
{ "txnId":"T101", "status":"SUCCESS",  "bankRef":"B778", "amount":500 }
{ "txnId":"T102", "status":"TIMEOUT",  "retry":1, "reason":"NO_BANK_RESPONSE" }
{ "txnId":"T101", "status":"REVERSAL", "refundRef":"R55", "reason":"CUSTOMER_REQUEST" }
```

2. Reverse-Engineered State Machine

Transition	From	Trigger	Data constraint	To
T1	Initiated	Submit payment	Valid amount > 0, valid account	Pending
T2	Pending	Bank approval	bankRef present	Success
T3	Pending	No response for 30 s	retry < 3	Timeout
T4	Timeout	Retry / status query	retry < 3	Pending
T5	Timeout	Retries exhausted	retry = 3	Reversal
T6	Success	Refund request	Within 24 h, amount <= paid	Reversal
T7	Pending	Bank decline	error code present	Reversal

3. Technique Identified

The behaviour is governed by **State Transition Testing** (finite state machine), with 0-switch coverage (every transition once) as the implicit minimum. Guard conditions (retry < 3, 24-hour window) add boundary value analysis on the data.

4. State-Transition Diagram

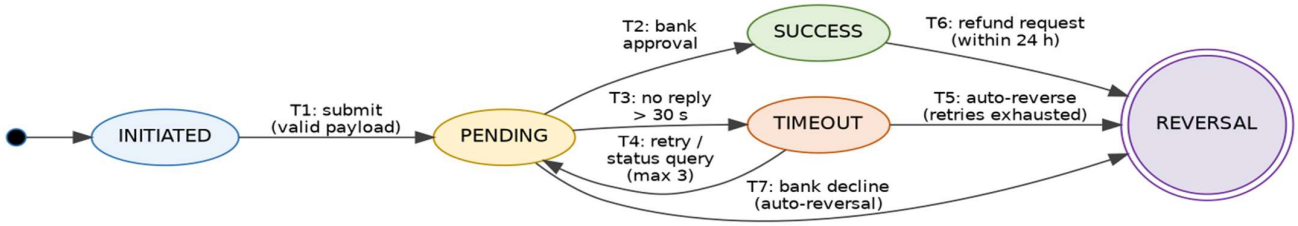


Figure 6.1 State-transition diagram of the payment gateway

5. State Table including Illegal Transitions

State \ Event	Submit	Approve	Timeout	Retry	Refund
Initiated	Pending	Illegal	Illegal	Illegal	Illegal
Pending	Illegal	Success	Timeout	Illegal	Illegal
Success	Illegal	Illegal	Illegal	Illegal	Reversal
Timeout	Illegal	Illegal	Illegal	Pending / Reversal	Illegal
Reversal	Illegal	Illegal	Illegal	Illegal	Illegal (terminal)

6. Reconstructed State-Based Test Suite

TC	Scenario	Type	Expected
1	Initiated to Pending (valid payload)	Legal T1	status = PENDING

TC	Scenario	Type	Expected
2	Pending to Success	Legal T2	bankRef returned
3	Pending to Timeout after 30 s	Legal T3	status = TIMEOUT
4	Timeout to Pending on retry 1	Legal T4	retry = 1
5	Three timeouts then Reversal	Legal T5 (guard boundary)	Auto refund
6	Success to Reversal at 23:59 h	Legal T6 (boundary)	Refund accepted
7	Success to Reversal at 24 h 01 min	Illegal (boundary)	Rejected
8	Pending to Reversal on bank decline	Legal T7	No debit
9	Success to Pending	Illegal	Error, state unchanged
10	Reversal to Success	Illegal	Error, terminal state
11	Initiated to Success (skip Pending)	Illegal	Rejected
12	Duplicate Approve on Success	Illegal / idempotency	No double credit
13	Refund amount > paid amount	Data constraint	Rejected
14	Late bank approval after Timeout	Race condition	Handled consistently (reverse or reconcile)
15	Path Initiated-Pending-Timeout-Pending-Success	1-switch sequence	Final Success

Coverage: all 7 legal transitions (100% 0-switch), all illegal cells of the state table, selected 1-switch sequences and guard boundaries.

Question 7: Encrypted Communication Module (Fuzzing and BVA)

1. Understanding of the System

The module accepts parameters (message, key, IV, algorithm) and returns ciphertext. Assumed fuzzing results on the input fields:

Field	Fuzz input	Observed behaviour	Inferred rule
Key	15, 16, 17 bytes	15 error, 16 OK, 17 error	Key lengths 16, 24, 32 bytes only
Message	0, 1, 1024, 1025 chars	0 error, 1 and 1024 OK, 1025 error	Length 1-1024
IV	15, 16, 17 bytes	Only 16 accepted	IV exactly 16 bytes
Message	' OR 1=1 --, <script>, %n%n	Encrypted as data / one case crashed	Missing sanitisation in one path
Message	100,000 chars	Crash / truncated output	No hard length guard before copy
Algorithm	null, "XYZ"	Error 400 / unhandled exception	Whitelist incomplete

2. Decomposition

Component	Function
Data validation layer	Type, length, character-set and null checks on all inputs
Parameter handler	Key and IV length rules, algorithm whitelist
Encryption engine	Block cipher, padding, output encoding
Error handler	Return controlled errors without leaking internals

3. Error-Handling Boundaries Identified

- Message length 1 to 1024 (valid), 0 and 1025 (invalid).
- Key length 16 / 24 / 32 bytes (valid), any other length invalid.
- IV length exactly 16 bytes.
- Algorithm must be in the allowed list.

4. Reconstructed BVA Test Cases

TC	Field	Value	Expected
1	Message length	0	Rejected, controlled error
2	Message length	1	Encrypted
3	Message length	2	Encrypted
4	Message length	1023	Encrypted
5	Message length	1024	Encrypted
6	Message length	1025	Rejected, no overflow
7	Key length	15 / 16 / 17	Reject / accept / reject
8	Key length	23 / 24 / 25	Reject / accept / reject
9	Key length	31 / 32 / 33	Reject / accept / reject
10	IV length	15 / 16 / 17	Reject / accept / reject
11	Message	65,536 and 1,000,000 chars (stress edge)	Rejected quickly, no crash
12	Message	SQL, script, format-string, null byte, Unicode, control characters	Treated as data or rejected, never executed

TC	Field	Value	Expected
13	Algorithm	null, empty, unknown name	Rejected with generic error
14	Same input twice	Same key, different IV	Different ciphertext (IV used correctly)
15	Round trip	Encrypt then decrypt, 1 and 1024 chars	Original message recovered

5. How the Design Prevents Buffer Overflow and Injection

- **Overflow:** tests at 1025 and far beyond (65,536; 1,000,000) confirm the length check occurs before any memory copy. Failure at these points would show a crash or memory corruption.
- **Injection:** payloads are verified to be treated purely as data, so they do not reach any query, shell or log formatter. Format-string (%n) cases check logging safety.
- **Information leakage:** invalid input must give generic errors with no stack trace or key material.
- **Recommended controls:** input length whitelist, strict type check, parameterised queries, safe string APIs, and fuzz tests in the regression suite.

Question 8: Insurance Premium Calculator (Branch and Path Coverage)

1. Understanding of the System

Legacy tests show 100% statement coverage but miss defects in multi-condition decisions. The logic is assumed to be:

```
premium = base;
if (age < 25 && accidents > 0)        premium *= 1.5;   // D1
if (age >= 60 || healthCond)          premium *= 1.3;   // D2
if (smoker && coverage > 500000)      premium += 5000;  // D3
return premium;
```

2. Reverse Engineering the Original Approach

A single test with every condition true (age 20, accidents 1, health condition, smoker, coverage 600,000) executes every statement, so statement coverage reaches 100%. But each decision was evaluated only as true, and the individual conditions were never evaluated as false. Therefore the original approach used **statement coverage only**, which is the weakest criterion.

Defect example	Why 100% statement coverage misses it
&& changed to || in D1	All-true test still gives true
age < 25 written as age <= 25	Boundary age never tested
coverage > 500000 written as >= 500000	Boundary never tested
D2 false path wrongly applies 1.3	False branch never executed

3. New Control Flow Graph

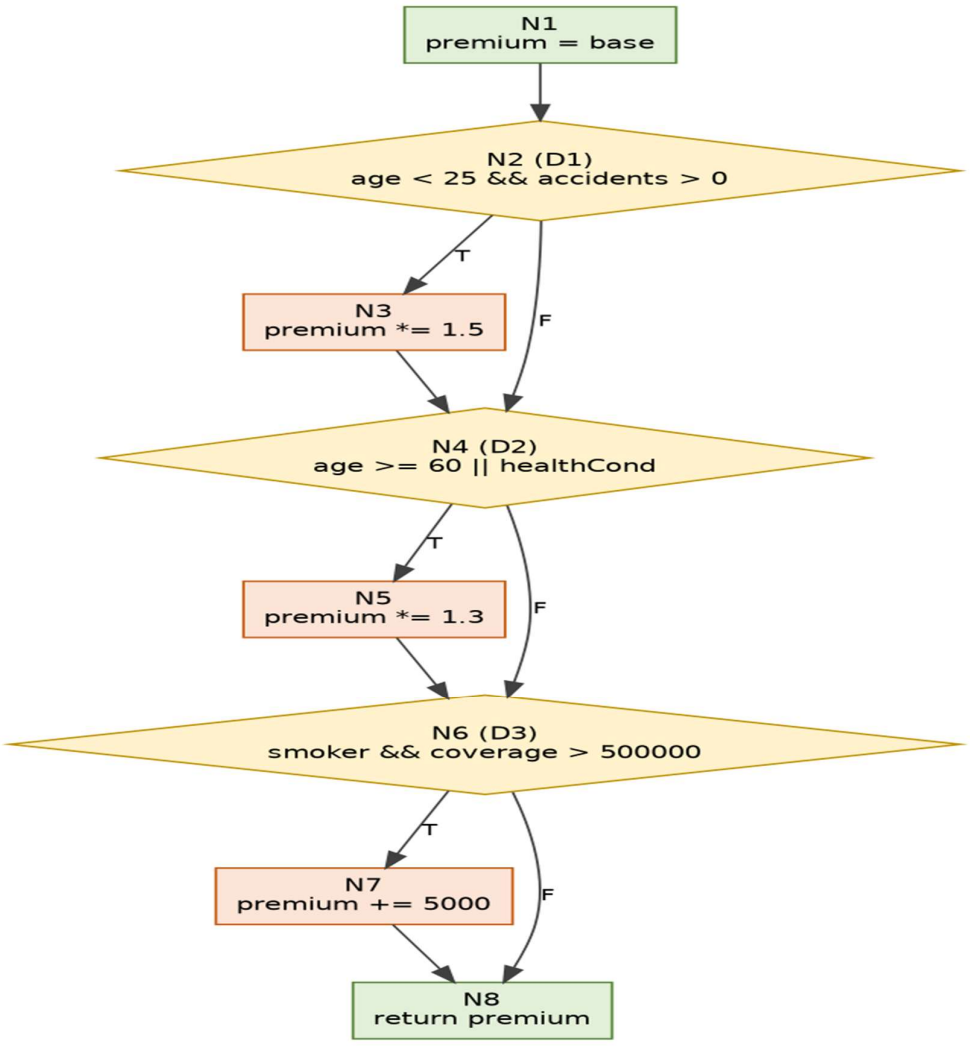


Figure 8.1 CFG of the premium calculator (three sequential decisions)

Edges E = 10, Nodes N = 8, so V(G) = E - N + 2 = **4** (3 decisions + 1). If each atomic condition is counted separately (6 conditions), V(G) = 7. Because the decisions are sequential and independent, the total number of distinct execution paths is 2 x 2 x 2 = **8**.

4. All Independent Execution Paths (base = 1,000)

Path	D1	D2	D3	Test data	Expected premium
P1	T	T	T	age 20, acc 1, health Y, smoker Y, cov 600000	6,950
P2	T	T	F	age 20, acc 1, health Y, smoker N	1,950
P3	T	F	T	age 20, acc 1, health N, smoker Y, cov 600000	6,500
P4	T	F	F	age 20, acc 1, health N, smoker N	1,500
P5	F	T	T	age 65, acc 0, health N, smoker Y, cov 600000	6,300
P6	F	T	F	age 65, acc 0, health N, smoker N	1,300
P7	F	F	T	age 40, acc 0, health N, smoker Y, cov 600000	6,000
P8	F	F	F	age 40, acc 0, health N, smoker N	1,000

5. Reconstructed Strategy for 100% Branch and Path Coverage

- **Branch coverage:** minimum two tests (all decisions true, all false) e.g. P1 and P8.
- **Path coverage:** all eight tests P1-P8 above.
- **Condition coverage / MC/DC** for compound decisions, with boundaries:

Decision	Condition tests (each condition shown to change outcome alone)
D1: age < 25 && accidents > 0	(24, 1) true; (24, 0) false; (25, 1) false
D2: age >= 60 || health	(60, N) true; (59, Y) true; (59, N) false
D3: smoker && coverage > 500000	(Y, 500001) true; (Y, 500000) false; (N, 500001) false

6. How This Reduces the Risk of Calculation Errors

- Every decision outcome and every combination of discounts is executed, so wrong operator precedence, wrong order of multiplication and addition, and missing branches are exposed.
- MC/DC with boundary values detects && versus || mix-ups and off-by-one comparison errors.
- Expected values are computed for all 8 combinations, so interaction errors between rules are caught.

Question 9: Cloud File Synchronization Tool (ECP, BVA and Stress Testing)

1. Understanding of the System

The tool syncs local files to the cloud. User activity logs and "lost file" defects are available. Assumed defect patterns:

Defect	File size	File type	Network	Outcome
L1	2.5 GB	Video	Fast	File missing in cloud
L2	2,147,483,648 B (2 GB)	Archive	Fast	Truncated / lost
L3	0 bytes	Any	Any	Never uploaded
L4	300 MB	Document	< 1 Mbps	Timeout, marked synced, file lost
L5	10 KB	.tmp / .pst / lock file	Fast	Silently skipped
L6	50 MB	Name with special / 260+ chars	Fast	Lost
L7	20 MB	Any	Connection drops mid-upload	Partial file treated as complete

2. Decomposition into Components

Component	Function	Related defects
File scanner	Detect new / changed files, filter by type and name	L3, L5, L6
Chunking / upload manager	Split large files, upload chunks	L1, L2
Network handler	Retry, timeout, resume	L4, L7
Integrity checker	Checksum and size verification	L4, L7
Sync status / logger	Mark file synced or failed	L4

3. Inferred Equivalence Class Partitioning by Developers

Factor	Probable classes used	Missed classes (cause of loss)
File size	Small (<10 MB), Medium (10 MB-1 GB)	Zero byte; at / above 2 GB (32-bit signed limit, 2^31)
File type	Documents, images, media	Temporary / system / locked files; unusual names and path length
Network speed	Fast (>10 Mbps)	Slow (<1 Mbps); intermittent; offline mid-transfer

The defects cluster just above the tested limits, which suggests developers tested typical values from each class but never the boundaries or the extreme classes.

4. Reconstructed Testing Strategy: BVA with Stress Testing

4.1 Boundary value analysis

Factor	Boundary test values
File size	0 B, 1 B, chunk size -1 / = / +1 (e.g. 4 MB), 2 GB - 1, 2 GB, 2 GB + 1, 4 GB + 1, max supported size and +1

Factor	Boundary test values
File name / path	1 char, 255 chars, 256 chars, 259 / 260 / 261 path chars, special and Unicode characters
Network speed	0 (offline), 0.5, 1, 1.1 Mbps, 10 Mbps; latency 1 s and 10 s
Timeout / retry	Just below, at and above timeout value; retry count max and max + 1

4.2 Combined ECP matrix

TC	Size	Type	Network	Expected
1	10 MB	Document	Fast	Synced, checksum equal
2	0 B	Document	Fast	Synced or clear user message
3	2 GB - 1 B	Video	Fast	Synced
4	2 GB	Video	Fast	Synced (no truncation)
5	2 GB + 1 B	Archive	Fast	Synced
6	300 MB	Document	0.5 Mbps	Resumes; not marked synced until verified
7	10 KB	.tmp / lock	Fast	Skip with log entry visible to user
8	50 MB	Name 256 chars / special	Fast	Synced or explicit error
9	20 MB	Any	Drop at 50% then restore	Resume, checksum match

4.3 Stress tests

- Many files at once (10,000 small files) and a mix of large files in parallel.
- Network flapping every few seconds during a large upload.
- Low disk space and low memory on the client; cloud storage quota full.
- File modified while uploading; same file edited on two devices (conflict).
- 24-hour soak test with continuous change events.

5. Justification: Prevention of Data Loss

- BVA at 2 GB and chunk edges removes the size-limit defects (L1, L2).
- Slow and interrupted network tests with checksum verification stop false "synced" status (L4, L7).
- Explicit tests for skipped types and long names ensure no silent loss (L5, L6): either synced or clearly reported.
- Stress tests expose race conditions and resource exhaustion that appear only in real usage. The key acceptance rule is: a file is never marked synced until its cloud checksum equals the local checksum.

Question 10: E-Commerce Search and Filter (Decision Table Testing)

1. Understanding of the System

Only SQL queries and result sets are available. Assumed query sequence:

```
Q1: SELECT * FROM products WHERE category = 'Shoes';
Q2: ... WHERE category = 'Shoes' AND price BETWEEN 1000 AND 3000;
Q3: ... WHERE category = 'Shoes' AND brand IN ('Nike','Adidas');
Q4: ... WHERE category = 'Shoes' AND stock > 0 AND rating >= 4;
Q5: ... WHERE (name LIKE '%run%' OR description LIKE '%run%') AND stock > 0;
Q6: ... WHERE category = 'Shoes' AND NOT (brand = 'Puma');
```

2. Reverse-Engineered Logic

Observation	Inferred logic
Different filters always joined by AND	Filters narrow results (intersection)
Multiple brands joined by IN / OR	OR inside one filter, AND between filters
Keyword searched in name and description	Keyword = name OR description
stock > 0 appears when "in stock" ticked	Boolean filter
NOT used for excluded brand	Exclusion filter
Price uses BETWEEN	Inclusive range; min <= max required

3. Decomposition

Component	Function
Parameter parser	Read and validate keyword, category, price range, brand list, stock flag, rating
Query builder	Build WHERE clause with AND / OR / NOT
Executor	Run parameterised SQL
Result formatter	Sort, paginate, "no results" message

4. Decision Table (5 binary conditions, 32 rules)

Conditions: Keyword entered, Category selected, Price range set, Brand selected, In-stock only. Y = filter applied, N = not applied.

Rule	Keyword	Category	Price	Brand	In stock	Expected result
R1	N	N	N	N	N	All products (no filter)
R2	N	N	N	N	Y	Products matching ALL of: in-stock
R3	N	N	N	Y	N	Products matching ALL of: brand
R4	N	N	N	Y	Y	Products matching ALL of: brand AND in-stock
R5	N	N	Y	N	N	Products matching ALL of: price
R6	N	N	Y	N	Y	Products matching ALL of: price AND in-stock
R7	N	N	Y	Y	N	Products matching ALL of: price AND brand
R8	N	N	Y	Y	Y	Products matching ALL of: price AND brand AND in-stock

Rule	Keyword	Category	Price	Brand	In stock	Expected result
R9	N	Y	N	N	N	Products matching ALL of: category
R10	N	Y	N	N	Y	Products matching ALL of: category AND in-stock
R11	N	Y	N	Y	N	Products matching ALL of: category AND brand
R12	N	Y	N	Y	Y	Products matching ALL of: category AND brand AND in-stock
R13	N	Y	Y	N	N	Products matching ALL of: category AND price
R14	N	Y	Y	N	Y	Products matching ALL of: category AND price AND in-stock
R15	N	Y	Y	Y	N	Products matching ALL of: category AND price AND brand
R16	N	Y	Y	Y	Y	Products matching ALL of: category AND price AND brand AND in-stock
R17	Y	N	N	N	N	Products matching ALL of: keyword
R18	Y	N	N	N	Y	Products matching ALL of: keyword AND in-stock
R19	Y	N	N	Y	N	Products matching ALL of: keyword AND brand
R20	Y	N	N	Y	Y	Products matching ALL of: keyword AND brand AND in-stock
R21	Y	N	Y	N	N	Products matching ALL of: keyword AND price
R22	Y	N	Y	N	Y	Products matching ALL of: keyword AND price AND in-stock
R23	Y	N	Y	Y	N	Products matching ALL of: keyword AND price AND brand
R24	Y	N	Y	Y	Y	Products matching ALL of: keyword AND price AND brand AND in-stock
R25	Y	Y	N	N	N	Products matching ALL of: keyword AND category
R26	Y	Y	N	N	Y	Products matching ALL of: keyword AND category AND in-stock
R27	Y	Y	N	Y	N	Products matching ALL of: keyword AND category AND brand
R28	Y	Y	N	Y	Y	Products matching ALL of: keyword AND category AND brand AND in-stock

Rule	Keyword	Category	Price	Brand	In stock	Expected result
R29	Y	Y	Y	N	N	Products matching ALL of: keyword AND category AND price
R30	Y	Y	Y	N	Y	Products matching ALL of: keyword AND category AND price AND in-stock
R31	Y	Y	Y	Y	N	Products matching ALL of: keyword AND category AND price AND brand
R32	Y	Y	Y	Y	Y	Products matching ALL of: keyword AND category AND price AND brand AND in-stock

Additional rules (conditions outside the 5-bit table)

Rule	Condition	Expected
X1	Price min > max	Validation error, no query run
X2	Price min = max	Products priced exactly at that value
X3	Several brands selected	OR between brands, AND with other filters
X4	Rating filter 4+ combined with any rule	Adds rating >= 4 as an AND
X5	Brand excluded (NOT)	Selected brand absent from results
X6	Combination returns no rows	"No results" message, no error
X7	Keyword with SQL characters (' ; --)	Treated as text (parameterised), no error
X8	Empty keyword with spaces only	Treated as no keyword
X9	Large result set	Pagination correct, count matches

5. Test Strategy

- One test case per rule R1-R32 with verified expected row sets from a controlled dataset containing items that satisfy and fail each filter.
- Dataset design: for each filter there are items that match only that filter, so any wrongly applied OR instead of AND is visible.
- Execute X1-X9 for validation, exclusion, security and empty-result behaviour.

6. Completeness of the Reconstructed Model

Aspect	Evaluation
Coverage	All 2^5 = 32 filter combinations covered, plus 9 extra rules
Strength	Verifies AND between filters, OR within brands and keyword, NOT exclusion
Limitation	Rating, sort order, pagination and multiple-value interactions are only sampled (full set would be 2^6 = 64 or more)
Assumption risk	Logic is inferred from limited queries; unobserved rules (e.g. synonyms, spell correction) may exist
Mitigation	Add pairwise testing for extra filters and confirm with product owner

Code of Conduct:

I **P.Paneendra (192321072)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:P.Paneendra

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation